ESTRUCTURA DE DATOS

Semana 4

Análisis de Problemas Complejos y Herramientas de Solución

Docente: Dr. Eric Gustavo Coronel Castillo

Curso:
SIST1201A — Ciclo 4°

Periodo:
2026-1 | Unidad I

Universidad Privada del Norte

C#

LOGRO DE LA SESIÓN

Al finalizar la sesión, el estudiante:

1. Aplica técnicas de análisis (Diagrama de Ishikawa, árbol de problemas) para descomponer problemas complejos de software.
2. Identifica causas raíz y propone soluciones algorítmicas estructuradas utilizando estructuras de datos enlazadas.
3. Implementa en C# soluciones que integran listas enlazadas simples y dobles para resolver problemas reales.
4. Evalúa la complejidad temporal y espacial de las soluciones propuestas.

REFLEXIÓN — Saberes Previos

Antes de iniciar, reflexiona sobre las siguientes preguntas:

1

¿Cómo decides qué estructura de datos utilizar cuando enfrentas un problema que aún no has visto antes?

💡 *Piensa en los criterios que usas para elegir entre un arreglo, una lista enlazada simple o doble.*

2

¿Qué técnicas o herramientas conoces para identificar la causa raíz de un problema antes de programar una solución?

💡 *¿Has usado diagramas, pseudocódigo, esquemas o algún otro método para analizar problemas complejos?*

REFLEXIÓN — Fundamento Teórico

1

Elección de estructura de datos

La selección de una estructura de datos depende de los patrones de acceso, frecuencia de inserción/eliminación y los requisitos de memoria. Cormen et al. (2022) establecen que «la elección de la estructura de datos adecuada es fundamental para la eficiencia de un algoritmo». Para problemas con acceso secuencial y tamaño dinámico, las listas enlazadas son superiores a los arreglos.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4.ª ed.). MIT Press.

2

Análisis de causa raíz en ingeniería de software

El análisis de causa raíz (RCA) es una técnica sistemática que permite identificar el origen de los problemas, no solo sus síntomas. Pressman & Maxim (2020) señalan que antes de codificar cualquier solución, el ingeniero debe aplicar técnicas como el Diagrama de Ishikawa, el Árbol de Problemas o los Cinco Por Qué (5 Whys) para garantizar que la solución aborda la causa y no el efecto visible.

Pressman, R. S., & Maxim, B. R. (2020). Ingeniería del Software: Un Enfoque Práctico (9.ª ed.). McGraw-Hill.

¿Qué es el Análisis de Problemas Complejos?

📌 **Definición**

Un problema complejo es aquel que involucra múltiples variables interdependientes, cuya solución no es inmediata ni trivial. En ingeniería de software, se manifiesta cuando los requisitos cambian dinámicamente o la estructura de datos subyacente impacta el rendimiento global del sistema (Cormen et al., 2022).

1. Comprender

🔍

Leer el problema con atención. Identificar entradas, salidas y restricciones. Reformular con tus propias palabras.

2. Descomponer

🧩

Dividir el problema en sub-problemas manejables (Divide y Vencerás). Aplicar árboles de problemas o mapas mentales.

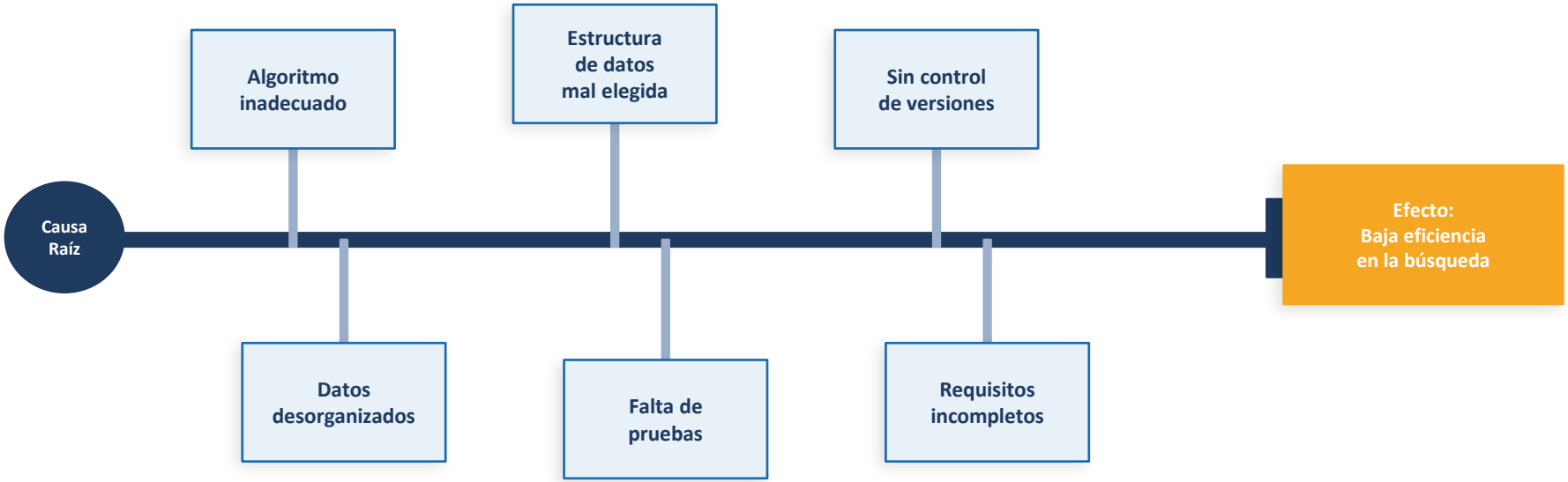
3. Modelar

🗂️

Elegir la estructura de datos adecuada. Representar la solución con diagramas UML o pseudocódigo antes de codificar.

Herramienta 1: Diagrama de Ishikawa (Causa–Efecto)

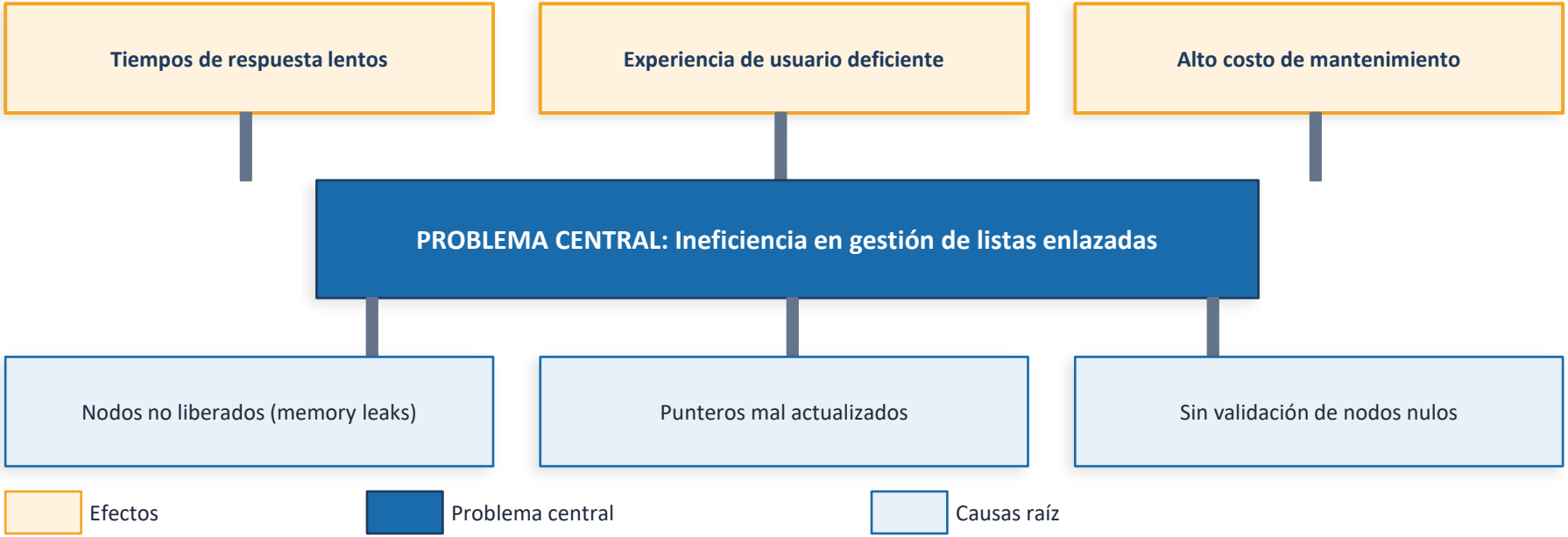
El Diagrama de Ishikawa (o Diagrama de Espina de Pescado) es una herramienta visual que permite identificar y organizar las causas potenciales de un problema (efecto). Fue desarrollado por Kaoru Ishikawa (1968) y es ampliamente adoptado en ingeniería de software para el análisis de defectos y problemas de rendimiento (Pressman & Maxim, 2020).



Pressman, R. S. & Maxim, B. R. (2020). Ingeniería del Software (9.ª ed.). McGraw-Hill. | Ishikawa, K. (1968). Guide to Quality Control. JUSE.

Herramienta 2: Árbol de Problemas

El Árbol de Problemas es una técnica participativa que representa visualmente un problema central, sus causas (raíces) y sus efectos (ramas). Se utiliza para priorizar intervenciones de diseño de software antes de implementar cualquier solución (ILPES/CEPAL, 2005).



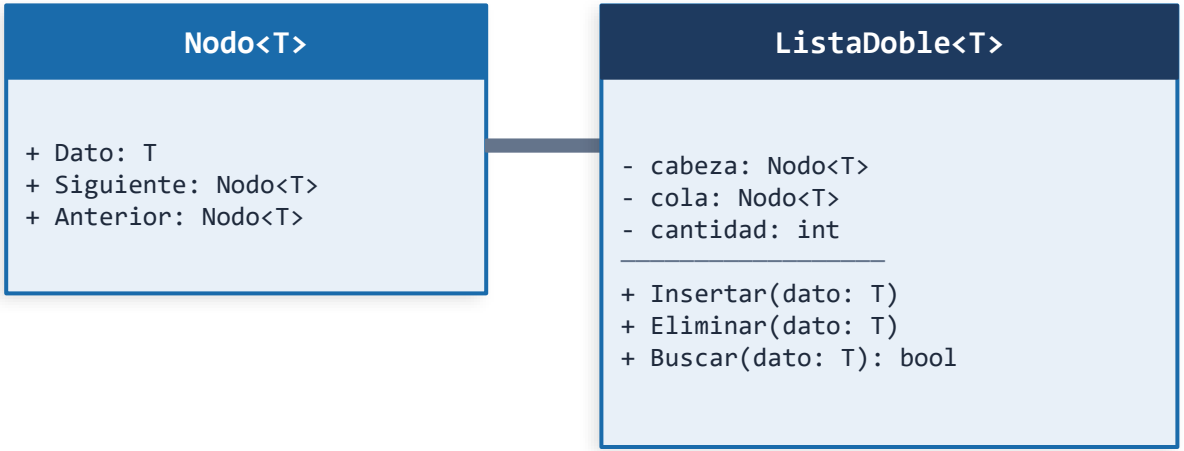
Herramienta 3: Los 5 Por Qué (5 Whys)

Técnica desarrollada por Sakichi Toyoda (Toyota, 1930). Consiste en preguntar «¿Por qué?» repetidamente —al menos cinco veces— hasta llegar a la causa raíz de un problema. Es especialmente útil en depuración de algoritmos y estructuras de datos (Liker, 2004).

Problema	El programa lanza NullReferenceException al recorrer la lista enlazada.
¿Por qué? 1	Porque el puntero 'siguiente' del último nodo no está inicializado en null.
¿Por qué? 2	Porque el método InsertarAlFinal() no establece nodo.Siguiente = null.
¿Por qué? 3	Porque no se implementó la condición de parada al crear nodos.
¿Por qué? 4	Porque no se aplicaron pruebas unitarias al método de inserción.
Causa raíz	Falta de pruebas unitarias y revisión del contrato del método de inserción.

De la Causa Raíz a la Solución: Diseño en C#

Diagrama de Clases (UML simplificado)



Implementación en C#

```
public class Nodo<T> {
  public T Dato;
  public Nodo<T> Siguiente;
  public Nodo<T> Anterior;
  public Nodo(T dato) {
    Dato = dato;
    Siguiente = null;
    Anterior = null;
  }
}

public class ListaDoble<T> {
  private Nodo<T> cabeza;
  private Nodo<T> cola;
  // Métodos: Insertar,
  // Eliminar, Buscar...
}
```

✅ Buena práctica (SOLID): Usar genéricos <T> en lugar de tipo object para garantizar seguridad de tipos en tiempo de compilación (Microsoft Learn, 2024).

Evaluación de Soluciones: Complejidad y Elección de Estructura

Al proponer una solución, el ingeniero debe evaluar la complejidad temporal O(n) y espacial O(1) de cada operación. La notación Big-O permite comparar la escalabilidad de las soluciones (Cormen et al., 2022).

Operación	Arreglo	Lista Simple	Lista Doble	Recomendación
Insertar al inicio	O(n)	O(1)	O(1)	**Lista Doble** ✅
Insertar al final	O(1)*	O(n)	O(1) con puntero cola	**Lista Doble** ✅
Buscar elemento	O(n)	O(n)	O(n)	Igual
Eliminar nodo	O(n)	O(n)	O(1) con puntero	**Lista Doble** ✅
Acceso aleatorio	O(1)	O(n)	O(n)	**Arreglo** ✅

** Amortizado con puntero al final.*

Proceso Integrado: Del Problema al Código

01

Entender el problema

Leer, identificar entradas/salidas, restricciones. Reformular con tus propias palabras.

02

Analizar causas

Aplicar Ishikawa, Árbol de Problemas o 5 Whys para identificar la causa raíz.

03

Diseñar la solución

Elegir la estructura de datos. Diagramar con UML. Escribir pseudocódigo.

04

Implementar en C#

Codificar siguiendo Clean Code y principios SOLID. Usar genéricos y nomenclatura PascalCase.

05

Verificar y refactorizar

Aplicar pruebas unitarias. Medir complejidad. Comparar con soluciones alternativas.

Martin, R. C. (2008). Clean Code. Prentice Hall. | Pressman & Maxim (2020). Ingeniería del Software. McGraw-Hill.

ACTIVIDAD EN CLASE — Ejercicio 1 de 4

Análisis de Causa Raíz con los 5 Por Qué

📋 Escenario

Un sistema de gestión hospitalaria usa una lista enlazada simple para registrar pacientes en cola de atención. Los médicos reportan que el sistema se congela cuando intentan eliminar un paciente del medio de la lista. El error mostrado es: 'System.NullReferenceException: Object reference not set to an instance of an object'.

📝 Instrucciones:

1. Aplica la técnica de los 5 Por Qué para identificar la causa raíz del error.

2. Completa la siguiente tabla en tu cuaderno o en el IDE con tu equipo (10 minutos).

3. Propón la solución en pseudocódigo antes de codificar.

Pregunta	Respuesta (completa en clase)
¿Por qué 1?	_____
¿Por qué 2?	_____

Modalidad: Trabajo en parejas

ACTIVIDAD EN CLASE — Ejercicio 2 de 4

Construye un Diagrama de Ishikawa

📋 **Problema:**

Una aplicación de biblioteca utiliza una lista doble enlazada para gestionar préstamos de libros. Al ejecutarse durante más de 2 horas, el rendimiento se degrada notablemente: las búsquedas que antes tardaban 10ms ahora tardan más de 500ms.

📝 **Instrucciones:**

Identifica al menos 2 causas para cada categoría:

Algoritmo	¿Qué operaciones son ineficientes? ¿Hay ciclos innecesarios?
Memoria	¿Se liberan correctamente los nodos eliminados? ¿Hay memoria no reclamada?
Código	¿Se usan recursividad o iteración excesiva? ¿Hay código duplicado?
Diseño	¿La estructura elegida es la más adecuada para búsquedas frecuentes?

Modalidad: Grupos de 3 estudiantes

ACTIVIDAD EN CLASE — Ejercicio 3 de 4

Implementa la solución en C#

📋 Enunciado

Implementa en C# una lista enlazada doble genérica ListaDoble<T> que incluya:

- Insertar(T dato): agrega al final.
- EliminarPorValor(T dato): elimina el primer nodo con ese valor, actualizando correctamente los punteros Anterior y Siguiente.
- Mostrar(): imprime todos los elementos en consola.

```
// Completa el código en Visual Studio

public void EliminarPorValor(T dato) {
  Nodo<T> actual = cabeza;
  while (actual != null) {
    if (actual.Dato.Equals(dato)) {
      // TODO: actualizar punteros
      // Anterior y Siguiente
      // ¿Qué pasa si es el primer
      // o el último nodo?
      return;
    }
    actual = actual.Siguiente;
  }
}
```

💡 Pistas:

1. Si el nodo a eliminar es la cabeza: cabeza = cabeza.Siguiente; y si la nueva cabeza no es null → cabeza.Anterior = null;
2. Si el nodo es la cola: cola = cola.Anterior; y si la nueva cola no es null → cola.Siguiente = null;

ACTIVIDAD EN CLASE — Ejercicio 4 de 4 (Reto)

🏆 Reto: Elige la mejor solución y justifícala

📋 Una empresa de delivery necesita gestionar pedidos en tiempo real. Los pedidos se insertan y eliminan constantemente desde ambos extremos (inicio y final). Además, ocasionalmente se requiere buscar un pedido por número de orden en el medio de la lista. Se manejan hasta 50,000 pedidos simultáneos.

Opción A: Arreglo dinámico (List<T>)

✅ Acceso aleatorio O(1)

⚠️ Inserción/eliminación al inicio O(n)

¿Elegiría esta opción? Justifica en 2 líneas.

Opción B: Lista Enlazada Simple

✅ Inserción al inicio O(1)

⚠️ Sin acceso directo al final O(n)

¿Elegiría esta opción? Justifica en 2 líneas.

Opción C: Lista Enlazada Doble

✅ Inserción/eliminación en extremos O(1)

⚠️ Mayor uso de memoria (2 punteros)

¿Elegiría esta opción? Justifica en 2 líneas.

Cormen et al. (2022) para referencia de complejidad.

CONCLUSIONES DE LA SESIÓN

El análisis precede al código

Aplicar herramientas como Ishikawa, Árbol de Problemas y los 5 Whys permite identificar la causa raíz antes de programar, evitando soluciones que solo atacan síntomas (Pressman & Maxim, 2020).

La estructura de datos determina la eficiencia

La elección entre lista simple, doble o arreglo debe basarse en los patrones de operación (inserciones, eliminaciones, búsquedas), evaluando la complejidad Big-O de cada caso (Cormen et al., 2022).

Clean Code y SOLID en la implementación

El uso de genéricos <T>, nomenclatura PascalCase y separación de responsabilidades (SRP) hacen el código mantenible, legible y escalable en el largo plazo (Martin, 2008).

Verificar, medir y refactorizar siempre

Una solución no termina al compilar. Medir tiempos, escribir pruebas unitarias y comparar complejidades son prácticas esenciales del ingeniero de software profesional.

¿Preguntas?

Tu participación enriquece el aprendizaje de todo el grupo.

eric.coronel@upn.edu.pe | Estructura de Datos — SIST1201A

Universidad Privada del Norte | Ingeniería de Sistemas Computacionales

REFERENCIAS BIBLIOGRÁFICAS

Normas APA 7.ª edición

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4.ª ed.). MIT Press.
   https://mitpress.mit.edu/9780262046305/

2. Pressman, R. S., & Maxim, B. R. (2020). Ingeniería del software: un enfoque práctico (9.ª ed.). McGraw-Hill Education.
   https://www.mheducation.com/highered/product/ingenieria-del-software-pressman-maxim/M9781456254094.html

3. Martin, R. C. (2008). Clean code: A handbook of agile software craftsmanship. Prentice Hall.
   https://www.pearson.com/us/higher-education/program/Martin-Clean-Code-A-Handbook-of-Agile-Software-Craftsmanship/PGM63937.html

4. Hernández Bejarano, M., & Baquero Rey, L. E. (2022). Estructuras de datos: fundamentación práctica. Editorial UPB.
   https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/230581

REFERENCIAS BIBLIOGRÁFICAS

Normas APA 7.ª edición

5 Liker, J. K. (2004). The Toyota way: 14 management principles from the world's greatest manufacturer. McGraw-Hill.

https://www.mheducation.com/highered/product/toyota-way-liker/M9780071392310.html

6 ILPES/CEPAL. (2005). Metodología del marco lógico para la planificación, el seguimiento y la evaluación de proyectos y programas. Naciones Unidas.

https://repositorio.cepal.org/handle/11362/5607

7 Microsoft. (2024). Genéricos (Guía de programación de C#). Microsoft Learn.

https://learn.microsoft.com/es-es/dotnet/csharp/fundamentals/types/generics

8 Microsoft. (2024). Linked list (clase System.Collections.Generic). Microsoft Learn.

https://learn.microsoft.com/es-es/dotnet/api/system.collections.generic.linkedlist-1

9 Algar Díaz, M. J., & Fernández de Sevilla Vellón, M. (2019). Introducción práctica a la programación con Python. eLibro.

https://elibro.bibliotecaupn.elogim.com/es/lc/upnorte/titulos/124259